Dissertation on

"Adaptive Study Planning for Self-Directed Learners"

Submitted in partial fulfilment of the requirements for the award of degree of

Master of Technology in

Data Science and Artificial Intelligence

Project Phase - 1

Submitted by:
Rohit Saji
PES2PGE24DS201

Under the guidance of
Prof. Ramesh Prakash Guledgudd
PES University

May – October 2026

M.TECH IN DATA SCIENCE AND ARTIFICIAL INTELLIGENCE
FACULTY OF ENGINEERING
PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

Certificate

This is to certify that the dissertation entitled "Adaptive Study Planning for Self-Directed Learners" is a bonafide work carried out by **Rohit Saji (PES2PGE24DS201)** in partial fulfilment for the completion of Project Phase - 1 in the Program of Study – Master of Technology in Data Science and Artificial Intelligence under the rules and regulations of PES University, Bengaluru.

Prof. Ramesh Prakash Guledgudd
PES University

Declaration

I hereby declare that the Project Phase - 1 entitled "Adaptive Study Planning for Self-Directed Learners" has been carried out under the guidance of Prof. Ramesh Prakash Guledgudd, and submitted in partial fulfilment of the course requirements for the award of the degree of Master of Technology in Data Science and Artificial Intelligence of PES University, Bengaluru, during the academic semester May – October 2026.

Acknowledgement

I would like to express my gratitude to my guide for continuous guidance and support throughout this project, and to the project coordinators for their help. I thank the Chairperson and faculty of the Department, PES University, for their support, and the university leadership for the opportunities provided. Finally, I extend heartfelt thanks to my family and friends for their constant encouragement.

Abstract

Self-directed learners routinely build fixed study plans that diverge from reality within days — they fall behind or move ahead — yet the plan never adapts, and learners have no reliable signal of how far off-track they are or whether they are genuinely learning. This work proposes an adaptive study-planning system that learns a learner's true pace, detects when their behaviour shifts, projects their completion date with calibrated uncertainty, and regenerates their schedule accordingly. The system is organised around two interlocking research pillars. The first, **closed-loop adaptation**, combines hierarchical Bayesian pace calibration, CUSUM change-point detection, and Gaussian-process progress projection with constraint-based schedule generation. The second, **assessment-based verification**, generates concept-tagged assessment items from the learner's own study material using large language models and applies cold-start knowledge tracing to estimate genuine concept mastery — providing an honest learning signal that guards against unreliable self-reported study data and feeds verified estimates back into calibration. Candidate algorithms for each component are compared offline against established baselines on a literature-grounded synthetic dataset with known ground truth and on public knowledge-tracing benchmarks, using detection latency, interval calibration, AUC, and schedule-adherence metrics. Phase I delivers and evaluates the open-loop system; Phase II closes the loop and introduces assessment-based verification as the principal novelty, evaluated by comparing closed-loop adaptive planning against a static open-loop baseline on schedule adherence and measured learning gain. The expected contribution is a verified closed-loop study-planning system that integrates learner calibration, behavioural-shift detection, uncertainty-aware projection, and assessment-grounded verification for self-directed learners.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter introduces the problem domain, the motivation for the project, the problem statement, the scope of the project, and the objectives to be achieved. It also provides an overview of the methodology followed and the structure of the report.

1.1 Problem Statement

Self-directed learners — people working through online courses, exam preparation, or self-study material without an instructor — typically begin with a fixed study plan ("two hours a day, finish by this date"). Within a few days the plan no longer matches reality: the learner is ahead or behind, the plan does not adjust, and there is no trustworthy signal of how far off-track they are or whether the time logged has produced genuine learning. Existing tools either track time without planning, or generate static plans that never adapt to the learner's realised pace, and none verify that self-reported study actually resulted in learning.

1.2 Objectives

1. Learn a learner's true study pace from their own session history using hierarchical Bayesian calibration.
2. Detect behavioural shifts in pace using statistical change-point detection.
3. Project completion with calibrated uncertainty using Gaussian-process regression.
4. Regenerate the study schedule using constraint-based, prerequisite-aware planning.
5. Verify genuine learning by generating concept-tagged assessments from the learner's own material and estimating mastery via cold-start knowledge tracing, closing the loop by feeding verified mastery back into calibration.
6. Evaluate the system against established baselines using synthetic data with known ground truth and public knowledge-tracing benchmarks.

1.3 Scope of the Project

The project is delivered across two phases. **Phase I** builds and evaluates the open-loop system — session logging, pace calibration, change detection, progress projection, and static schedule generation — and researches candidate approaches for assessment-based verification. **Phase II** closes the loop and implements the chosen assessment-

based verification approach as the principal novelty. The target user is the individual self-directed learner; multi-user and institutional features are out of scope.

1.4 Organization of the Report

Chapter 2 surveys related work across both research pillars and identifies the gaps. Chapter 3 specifies system requirements. Chapter 4 describes the proposed methodology, system architecture, algorithms, and expected outcomes. Chapters 5–7 cover implementation, results, and conclusions, and are developed across the project phases.

Chapter 2

Literature Survey

This chapter surveys the research relevant to the project and identifies the gaps it addresses. The work spans two research pillars, and the survey is organised accordingly. Section 2.1 covers **Pillar A — adaptive pace modelling and scheduling**: learner pace modelling and Bayesian calibration (§2.1.1), behavioural-change detection (§2.1.2), progress prediction with Gaussian processes (§2.1.3), adaptive scheduling and study-plan generation (§2.1.4), and self-regulated-learning analytics (§2.1.5). Section 2.2 covers **Pillar B — assessment generation, knowledge tracing, and verification**: knowledge tracing and the cold-start problem (§2.2.1), automatic question generation with large language models (§2.2.2), concept extraction and tagging (§2.2.3), automated answer evaluation (§2.2.4), assessment integrity and gaming detection (§2.2.5), and retention and review scheduling (§2.2.6). Section 2.3 summarises the approaches in a comparison table, Section 2.4 lists the limitations of existing systems, and Section 2.5 states the proposed solution.

2.1 Existing Systems

2.1.1 Pillar A: Adaptive Pace Modelling and Scheduling

2.1.1.1 Learner Pace Modelling and Bayesian Calibration

Estimating how fast a learner truly works, from limited per-learner data, is the foundation of the adaptation pillar. Chen et al. [2] propose a hierarchical Bayesian model of adaptive teaching and validate it through two online behavioural experiments with 312 adults, showing that hierarchical priors capture learner variability without overfitting to sparse data — the statistical justification for estimating a single learner's pace from few sessions. Zanellati et al. [3] present a systematic review of hybrid knowledge-tracing models in the *IEEE Transactions on Learning Technologies*, building a taxonomy of approaches that combine Bayesian posteriors with machine learning and concluding that hybrid models consistently outperform either pure Bayesian Knowledge Tracing or pure deep learning. Zambrano et al. [4] analyse algorithmic bias in Bayesian Knowledge Tracing at scale and report that BKT performance is essentially equal across demographic groups, supporting the reliable use of BKT-family models across diverse learners. Complementing these, Mai et al. [14] propose a Transformer–Bayesian hybrid network that couples deep temporal modelling with a probabilistic layer to recover interpretable, calibrated knowledge estimates, reinforcing the finding that hybrid Bayesian designs retain

transparency without sacrificing predictive accuracy. These works establish Bayesian calibration as sound for learner modelling but address correctness prediction rather than the planned-versus-actual *pace* estimation this project requires.

2.1.1.2 Behavioural-Change Detection

A learner's pace is not stationary; detecting when it shifts is essential to timely replanning. Saqr et al. [5] apply critical-slowing-down (CSD) indicators in an idiographic analysis of 1.67 million practice attempts from 9,401 students and find that 88.2% of disengaging learners exhibit detectable warning signals beforehand — strong evidence that per-learner behavioural shifts are detectable from trace data. Qiao et al. [6] provide a PRISMA systematic review of statistical-process-control methods, documenting the mathematical rationale for CUSUM and EWMA control charts and their machine-learning-enhanced variants, although all reviewed applications target manufacturing rather than education. Boca de Giuli et al. [7] combine control charts with continual learning to distinguish endogenous system shifts from one-time exogenous disruptions — a distinction that maps directly onto the project's need to recalibrate on genuine pace change while ignoring one-off disruptions. Closer to the educational setting, Cheng et al. [15] model behavioural change directly from heterogeneous student records — assignments, grades, and attendance — for early at-risk prediction, deliberately retaining qualitative information that pure quantitative feature extraction discards; this is evidence that behavioural-shift signals are recoverable from the same kind of trace data this project logs. The gap is that these methods detect shifts but prescribe no corrective action.

2.1.1.3 Progress Prediction with Gaussian Processes

Projecting a completion date *with* a calibrated uncertainty band, rather than a single number, is what makes a projection trustworthy. Pérez-Suay et al. [8] apply Gaussian-process (GP) regression with an automatic-relevance-determination kernel to learning-management-system logs, achieving $R = 0.89$ for predicting student marks with interpretable feature weighting. Chen and Yao [9] combine black-hole optimisation for feature selection with a weighted GP-regression ensemble, reporting RMSE 0.95 and MAE 0.81 and showing that GP predictions degrade gracefully with limited data. Lin et al. [10] introduce a latent Kronecker structure to scale GPs for learning-curve prediction, matching Transformer performance while retaining calibrated uncertainty. Beyond regression, Gaussian-process *classification* with meta-heuristic hyper-parameter search has been used to predict student performance bands [16], and integrated multi-modal machine-learning frameworks report strong engagement- and dropout-prediction accuracy on educational data [17] — indicating that probabilistic and ensemble predictors generalise across educational prediction tasks. These works validate GP regression on educational data, but apply it cross-sectionally rather than to the longitudinal burn-up projection the project needs.

2.1.1.4 Adaptive Scheduling and Study-Plan Generation

The system must turn estimates into a concrete, regenerated schedule. Islam et al. [1] couple an artificial neural network for grade prediction (accuracy ≈ 0.73) with dynamic programming for optimal schedule allocation, establishing the *predict-then-optimize* architecture this project adopts and extends; it is taken as the base paper for Pillar A.

Fabregar and Amorado [12] generate expert-quality study plans in 0.32 s using a greedy algorithm with prerequisite-constraint satisfaction, demonstrating fast constraint-based, prerequisite-aware scheduling. Pagano et al. [11] report a controlled study of 2,064 students showing that adaptive learning paths produce substantial efficiency gains over static ones — both large-scale evidence that adaptation works and the template for an adaptive-versus-static evaluation design. Knowledge-graph-based planners take a complementary route: Liu et al. [18] integrate a domain knowledge graph with AI-driven sequencing to generate personalised learning paths that respect concept prerequisites, illustrating prerequisite-aware path construction at the curriculum level. The limitation across these works is that the generated plan is static: it never adapts to the learner's realised pace.

2.1.1.5 Self-Regulated-Learning Analytics

Alhazbi et al. [13] review 25 empirical studies on measuring self-regulated learning (SRL) from trace data and find that most validated indicators relate to time-management skills, grounding the project's choice of metrics — schedule adherence and regularity — in established SRL frameworks. de Barba et al. [19] develop and validate an SRL learning-analytics rubric that maps validated SRL scales onto learning-management-system data indicators in a scalable, explainable way, providing a template for turning raw trace data into interpretable self-regulation signals. Uzun et al. [20] show empirically that learners' engagement with analytics feedback is associated with both SRL competence and course performance, motivating the project's feedback-oriented design. More broadly, hybrid machine-learning and rough-set models have been applied to identify at-risk learners and trigger personalised interventions [21], situating adherence analytics within a wider student-management context. These works supply the validated indicators but stop at measurement; none feeds the SRL signal back into an adaptive plan.

2.1.2 Pillar B: Assessment Generation, Knowledge Tracing, and Verification

2.1.2.1 Knowledge Tracing and the Cold-Start Problem

Knowledge tracing (KT) estimates a learner's evolving mastery from their response history and is the mechanism this project uses to convert assessment outcomes into a verified learning signal. The cold-start work of [22] (CLST) observes that most KT models are ID-based and perform poorly for new learners with little history, and mitigates this by aligning a generative large language model as a student knowledge tracer; it is taken as the base paper for Pillar B because single-learner, few-session cold-start is precisely the project's setting. MAML-KT [23] frames new-student prediction as few-shot model-agnostic meta-learning, showing that standard models (DKT, DKVMN, SAKT) exhibit substantially lower early accuracy than reported once the cold-start scenario is made explicit. DKVMN&MRI [24] extends the Dynamic Key-Value Memory Network with multi-relational information (exercise–knowledge-component and exercise–exercise relations) to address sparse input and weak interpretability. Multi-concept modelling is directly relevant here because assessment items rarely test a single concept: SLAM [36] models sequential learning signals across multiple concepts per question, relaxing the single-concept assumption that limits many KT models, while [37] integrates question-level information with learner behaviour to sharpen mastery estimates. The concept-driven model of [25] adds a *personalised* forgetting mechanism, modelling forgetting per

learner and across related concepts rather than as a single time-decay — the closest prior work to this project's concept-level, retention-aware mastery model. Graph-based KT for MOOC path recommendation [26] models heterogeneous relations among students, learning objects, and knowledge components and connects mastery estimation to next-activity recommendation, bridging Pillar B back to scheduling.

2.1.2.2 Automatic Question and Item Generation with Large Language Models

The verification pillar must first generate assessments from the learner's own material. The systematic study of [27] investigates LLM prompting for automatic item generation across STEM subjects and characterises both the capability and the risks — hallucination and the promotion of misconceptions — that any generation pipeline must guard against. The self-hosted lecture-to-quiz pipeline of [28] converts lecture PDFs into multiple-choice questions using a *local* LLM with deterministic quality control, keeping content on-device and emitting auditable question banks; this is close prior art for the project's ingest-material-to-assessment pipeline on a self-hosted service. A growing body of work refines this capability for assessment specifically. Nikolovski et al. [38] use LLM-based agents to align generated questions with course objectives and Bloom's-taxonomy cognitive levels, improving clarity and difficulty control. Recognising that fully automatic generation risks factual errors and uneven difficulty, Burke [39] proposes a human-in-the-loop framework for exam-variant generation that retains template-level scalability while inserting expert review, and Alamoudi et al. [40] combine LLMs with ontologies to optimise automated question generation, reporting gains in semantic-similarity and BERT-based quality metrics over flat-prompt baselines. Together these establish both the promise and the quality-control discipline the project's generation pipeline must adopt.

2.1.2.3 Concept and Knowledge-Component Extraction

For knowledge tracing to operate over freshly generated items, each item must be tagged with the concept it assesses. The work of [29] uses GPT-4 to generate and tag knowledge components for multiple-choice questions in Chemistry and E-Learning and validates them against three domain experts per subject, quantifying the human–LLM discrepancy. Complementarily, [30] automates knowledge-concept tagging on mathematics questions using LLMs with a flexible demonstration retriever, replacing manual expert annotation. Recent work frames this tagging as Q-matrix construction: Lopes and Netto [41] evaluate LLM-generated item–knowledge-component Q-matrices both structurally and functionally, treating the matrix as an interpretable item–attribute mapping that a tracing model can consume directly, and Hendrawan et al. [42] give a data-driven evaluation of LLM-based ontology concept extraction from programming content, quantifying where automated extraction succeeds and fails on the project's own domain. On the consuming side, KeenKT [43] disambiguates a learner's mastery state — separating genuine ability from an outburst or carelessness via a Normal-Inverse-Gaussian representation — which matters when mastery must be inferred from a small number of freshly generated items. Together these establish the concept-tagging step that links generated items to the mastery model.

2.1.2.4 Automated Answer Evaluation and Grading

Generated items must be graded automatically across the project's theory and coding modalities. The work of [31] applies LLMs to grade open-ended short answers in software-engineering courses, where many distinct answers may be partially correct, demonstrating automated evaluation beyond pure code. The empirical study of [32] evaluates ChatGPT as an automated essay-scoring tool using criteria-grounded prompts derived from the official TOEFL guide, situating LLM grading against established deep-learning and statistical scoring methods. For the coding modality specifically, Jukiewicz [44] reports a large-scale, side-by-side comparison of contemporary LLMs grading over 6,000 programming submissions, characterising the consistency and leniency differences any automated coding-grader must account for. On the theory side, rubric-grounded LLM grading of short answers [45] and comparative analyses of LLMs on open-ended mathematics responses [46] confirm that automated evaluation extends across free-text and numeric answers alike, while underscoring the residual variability that argues for rubric anchoring and human spot-checks.

2.1.2.5 Assessment Integrity and Gaming Detection

The motivation for verification is that self-reported activity can be gamed. The think-aloud study of [33] analyses gaming the system through the lens of self-regulated learning, moving beyond a purely behavioural view to ask whether students are cognitively disengaged — directly relevant to framing the "fake session" problem. The multi-stage system of [34] detects online-assessment misconduct by combining internet-protocol/geographical verification with behavioural analysis, illustrating practical integrity safeguards for online assessment. Beyond protocol-level checks, deep-learning and classical machine-learning models have been trained to flag anomalous exam behaviour directly from interaction data [47], demonstrating data-driven cheating detection that complements the project's synthetic honest-versus-faker evaluation.

2.1.2.6 Retention and Review Scheduling

Finally, the system must decide *when* to assess. Personalised spaced-repetition scheduling [35] schedules review at algorithmically chosen intervals to maximise retention, informing the placement of assessments and reviews within the generated roadmap; the personalised forgetting model of [25] provides the complementary per-concept retention estimate. Closing the loop between generation and retention, recent work uses LLMs to generate *retrieval-practice* items scheduled at spaced intervals [48], directly linking the project's two Pillar B capabilities — item generation and review timing — within a single retention-oriented pipeline.

2.1.3 Summary of Approaches

Table 2.1 summarises the principal approach families across both pillars with their strengths and the limitations that motivate this work. The recurring pattern is that each technique is strong in isolation but disconnected from the others: none forms a closed loop, and none uses assessment to verify the learning signal that drives planning.

Table 2.1: Summary of approach families across the two pillars, with strengths and limitations.

Approach family	**Strengths**	**Limitations for this project**
Bayesian calibration [2, 3, 4, 14]	Works with sparse, per-learner data; interpretable; fair across groups	Targets correctness, not planned-vs-actual pace
Change-point detection [5, 6, 7, 15]	Detects behavioural shifts early; well-founded (SPC)	Detects only; prescribes no corrective replanning
Gaussian-process projection [8, 9, 10, 16, 17]	Calibrated uncertainty; degrades gracefully with little data	Applied cross-sectionally, not to longitudinal burn-up
Adaptive scheduling [1, 12, 11, 18]	Fast, prerequisite-aware; adaptation shown to help	Generated plan is static; no feedback from realised pace
SRL analytics [13, 19, 20, 21]	Validated trace-based indicators of self-regulation	Measures behaviour; not tied to adaptive replanning
Knowledge tracing [22, 23, 24, 25, 26, 36, 37, 43]	Estimates mastery; cold-start and forgetting addressed	Assumes fixed item bank / many learners
LLM item generation [27, 28, 38, 39, 40]	Generates assessments from raw material; self-hostable	Hallucination risk; decoupled from the learner's plan
Concept tagging [29, 30, 41, 42]	Automates KC linking of items	Studied apart from tracing and planning
Automated grading [31, 32, 44, 45, 46]	Scales evaluation across theory and code	Not connected to mastery estimation or scheduling
Integrity / gaming [33, 34, 47]	Surfaces dishonest engagement	Studied separately from planning
Retention scheduling [35, 48]	Times reviews to maximise retention	Not integrated with verified mastery

2.2 Limitations of Existing Systems

1. **Techniques exist in isolation.** Calibration, change detection, projection, and scheduling are each studied alone; no system integrates them into a single connected loop.
2. **No closed feedback to the schedule.** Behavioural monitoring [5] detects shifts but prescribes no corrective replanning; scheduling work [1, 12] is static and does not adapt to realised pace.
3. **No uncertainty-aware projection for self-directed learners.** GP work [8, 10] is cross-sectional; calibrated learning-curve projection is not applied to ongoing self-directed study.
4. **Self-reported study activity is unverified.** No planning system uses assessment to confirm that logged study produced learning; gaming and self-report validity [33, 34] are studied separately from planning.
5. **Knowledge tracing assumes fixed item banks and many learners** [24, 25]; even cold-start work [22, 23] is not adapted to LLM-generated, per-learner, concept-tagged items in a single-learner setting.
6. **Assessment generation, tagging, grading, and tracing are decoupled** [27, 28, 29, 30, 31] from the learner's plan — generated items are not used to drive calibration or rescheduling.

2.3 Proposed Solution

This project proposes a **verified closed-loop** adaptive study-planning system for self-directed learners, organised around two interlocking pillars that together close the gaps above. **Pillar A (closed-loop adaptation)** extends the predict-then-optimize architecture [1] into an adaptive loop: hierarchical Bayesian pace calibration [2], change-point detection of behavioural shifts [5, 6], Gaussian-process progress projection with calibrated uncertainty [8], and constraint-based, prerequisite-aware schedule regeneration [12], evaluated with the adaptive-versus-static paradigm [11] and self-regulated-learning metrics [13].

Pillar B (assessment-based verification) extends cold-start knowledge tracing [22, 23] to concept-level mastery over LLM-generated, concept-tagged items [25, 28, 29, 30]: the system ingests the learner's own material, generates assessments [27], grades them [31], and traces genuine mastery — an honest learning signal that guards against unverified self-reported study [33] and, in Phase II, feeds back into calibration to close the loop. Review placement draws on spaced-repetition scheduling [35]. Phase I delivers and evaluates the open-loop system; Phase II closes the loop and introduces assessment-based verification as the principal novelty — to the best of this survey's knowledge, the first study-planning system to integrate learner calibration, behavioural-shift detection, uncertainty-aware projection, and assessment-grounded verification for self-directed learners.

Chapter 3

System Requirements Specification

This chapter covers the system requirements, including hardware and software specifications necessary for the project.

3.1 Hardware Requirements

- Commodity laptop/desktop (8 GB RAM or higher); no dedicated GPU required — per-learner training data is small and models train on CPU.
- Local container runtime for the intelligence service (Docker via Colima).

3.2 Software Requirements

- **Client:** React + Vite (TypeScript), Dexie (IndexedDB) for local-first storage.
- **Intelligence service:** Python, FastAPI, Docker; scientific stack (NumPy, SciPy, scikit-learn) and knowledge-tracing tooling (pyKT, pyBKT).
- **Backend services:** Supabase (authentication, event database, storage).
- **External:** large-language-model API for assessment generation and grading.

3.3 Functional Requirements

The system shall log study sessions, calibrate learner pace, detect pace shifts, project completion, generate and regenerate schedules, and (Phase II) generate assessments and estimate mastery.

3.4 Non-Functional Requirements

The core logging loop shall function offline; analytical results shall be cached locally; the system shall scale to the individual-learner workload without specialised hardware.

Chapter 4

Proposed Methodology

This chapter explains the overall methodology, system architecture, algorithms, and the expected outcomes against which the system will be evaluated.

4.1 System Architecture

The system follows a three-tier design (Figure 4.1): a local-first client, a Python intelligence service that hosts all heavy machine-learning components, and an offline research pipeline that produces the trained models the service runs. Supabase acts as the data hub between client and service. Pillar A (adaptation) runs as an open loop in Phase I; Pillar B (verification) and the closed feedback loop — in which verified mastery is fed back into pace calibration — are introduced in Phase II.

4.2 Algorithm and Model Description

Pillar A comprises four components: (i) hierarchical Bayesian *pace calibration* estimating the ratio of actual to planned effort at global, per-role, and per-context levels; (ii) CUSUM *change-point detection* over the pace-ratio time series; (iii) Gaussian-process *progress projection* producing a completion estimate with a 95% confidence interval; and (iv) constraint-based *schedule generation* with role-based material phasing. **Pillar B** comprises (i) LLM-based *item generation* producing concept-tagged assessment items from ingested material; (ii) *answer evaluation* for theory and coding responses; and (iii) cold-start, concept-level *knowledge tracing* that estimates mastery and, in Phase II, feeds the verified signal back into calibration.

4.3 Expected Outcomes and Evaluation Methodology

Evaluation rests on three legs so that single-user data is not load-bearing: (1) **synthetic data with known ground truth** (planted pace, regime shifts, and honest/faker labels) for precise algorithm comparison; (2) **public benchmarks** (knowledge-tracing datasets) for real-data credibility; and (3) a **closed-loop versus open-loop** comparison (Phase II) with the candidate's own logged sessions as longitudinal validation. Outcomes are stated as named metrics with directional hypotheses; firm numbers follow once experiments are run.

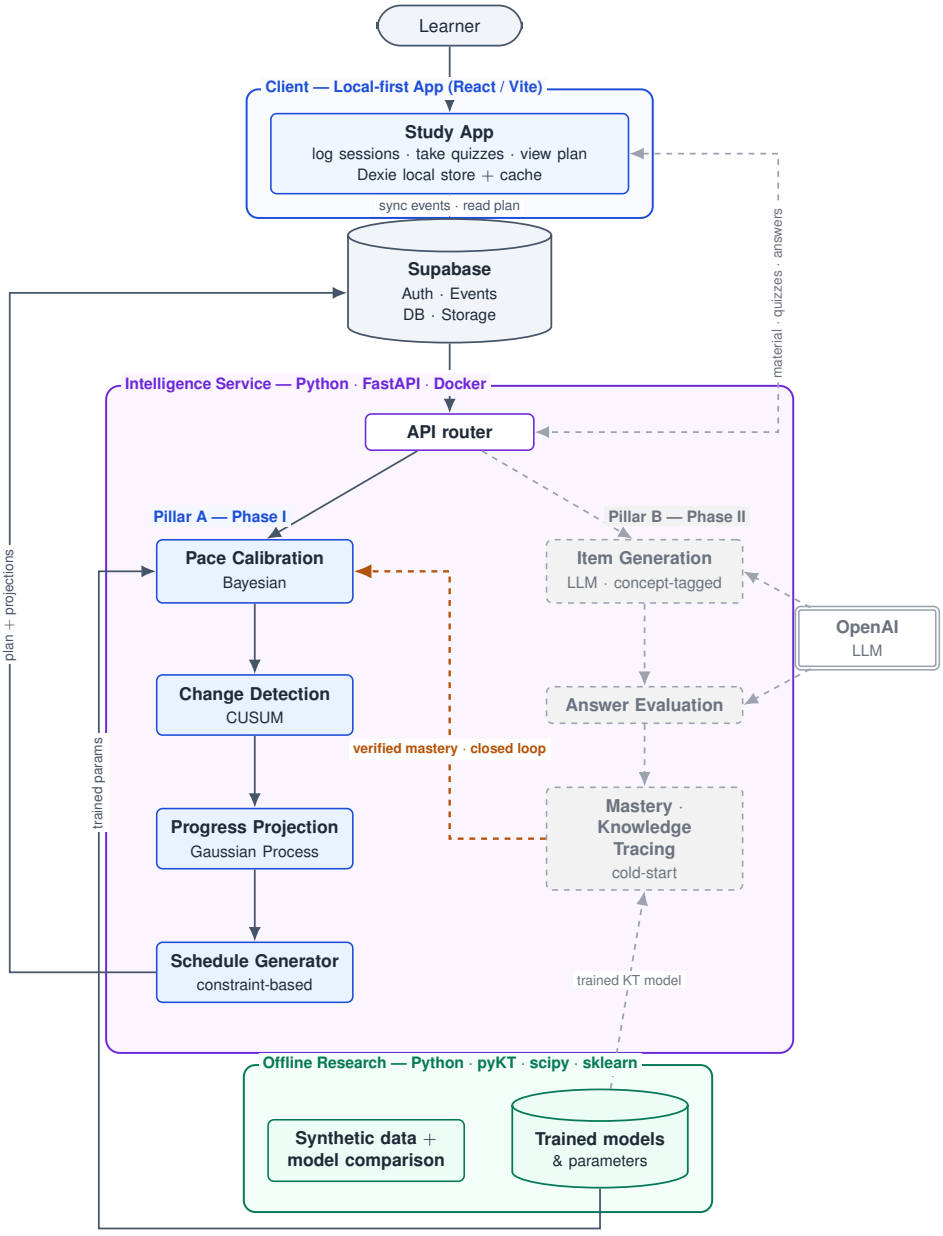

Figure 4.1: System architecture. Solid blue elements (Pillar A: Bayesian pace calibration, CUSUM change detection, Gaussian-process projection, constraint-based scheduling) run as an open loop in Phase I; dashed grey elements (Pillar B: LLM item generation, answer evaluation, cold-start knowledge tracing), the OpenAI dependency, and the verified-mastery feedback edge are introduced in Phase II. Supabase is the data hub; an offline research pipeline supplies the trained models the engines run.

Table 4.1: Expected outcomes — Pillar A (offline algorithm comparison).

Component	**Metric**	**Compared against**	**Expected (directional)**
Pace calibration	MAE / RMSE of predicted vs. actual pace	Hierarchical Bayesian vs. moving average	Bayesian lower error, especially with sparse data
Change detection	Detection latency, false-alarm rate	CUSUM vs. EWMA vs. CSD	A clear latency/false-alarm winner per profile
Progress projection	95% CI coverage, point error	Gaussian process vs. linear extrapolation	GP gives calibrated intervals; linear does not
Schedule generation	Deadline drift, schedule adherence	Constraint-based vs. rule-based vs. DP	Constraint-based competitive and prerequisite-aware

Table 4.2: Expected outcomes — Pillar B (public benchmarks and synthetic honest/faker data).

Component	**Metric**	**Compared against**	**Expected (directional)**
Mastery model	AUC on public KT datasets	DKT, BKT baselines	Competitive AUC, $\approx$ 0.70–0.82 (literature range)
Cold-start mastery	AUC at few interactions	Cold-start vs. standard KT	Cold-start higher AUC in low-data regime
Verification	Precision / recall of fake-session flags	Synthetic honest-vs-faker data	High precision without over-flagging honest learners
Item generation	Item validity / answerability rate	Human-checked sample	Majority valid; failure modes characterised
Learning signal	Pre/post mastery delta	—	Detectable gain after genuine study

4.4 Workflow Diagram

Chapter 5

Implementation Details

This chapter provides details of the development process, programming languages, tools and libraries, dataset descriptions, preprocessing, model building, and evaluation metrics.

5.1 Development Environment

5.2 Dataset Description

5.3 Model Implementation

5.4 Evaluation Metrics

Chapter 6

Results and Discussion

This chapter presents the experimental results obtained after testing the models, including performance graphs, metric scores, and a comparative discussion with existing methods.

6.1 Result Analysis

Chapter 7

Conclusion and Future Work

This chapter summarizes the outcomes of the project, the conclusions drawn, and suggests directions for future work.

7.1 Conclusion

7.2 Future Work

Bibliography

[1] Islam, M., et al., "Predicting Student Performance and Optimizing Study Schedule using Deep Learning and Dynamic Programming," *Proc. International Conference on Computer and Information Technology (ICCIT)*, IEEE, 2024.

[2] Chen, et al., "A Hierarchical Bayesian Model of Adaptive Teaching," *Cognitive Science*, 2024.

[3] Zanellati, A., et al., "Hybrid Models for Knowledge Tracing: A Systematic Review," *IEEE Transactions on Learning Technologies*, 2024.

[4] Zambrano, A. F., et al., "Investigating Algorithmic Bias in Bayesian Knowledge Tracing," *Proc. Learning Analytics and Knowledge (LAK)*, ACM, 2024.

[5] Saqr, M., et al., "Early Warning Signals Before Dropping Out: Critical Slowing Down Indicators," *Proc. Learning Analytics and Knowledge (LAK)*, ACM, 2026.

[6] Qiao, et al., "Machine-Learning Approaches for Statistical Process Control: A Systematic Review," *Entropy*, 2026.

[7] Boca de Giuli, et al., "Monitoring and Continual Learning of Dynamical Models," *Proc. European Control Conference (ECC)*, IEEE, 2024.

[8] Pérez-Suay, A., et al., "Predicting Student Performance from Moodle Logs with Gaussian Processes," *IEEE-RITA*, 2024.

[9] Chen, and Yao, "Predicting Academic Achievement with Black-Hole-Optimized Gaussian Process Regression," *Scientific Reports*, 2025.

[10] Lin, et al., "Scaling Gaussian Processes for Learning-Curve Prediction with Latent Kronecker Structure," arXiv preprint, 2024.

[11] Pagano, et al., "Efficiency of Adaptive Learning Paths," *Interactive Technology and Smart Education*, 2026.

[12] Fabregar, and Amorado, "A Study Plan Recommender Based on a Greedy Algorithm," *Proc. International Conference on Electrical Engineering and Informatics (EECSI)*, IEEE, 2025.

[13] Alhazbi, S., et al., "Using Learning Analytics to Measure Self-Regulated Learning: A Systematic Review," *Journal of Computer Assisted Learning*, 2024.

[14] Mai, N. T., Cao, W., Liu, W., "Interpretable Knowledge Tracing via Transformer–Bayesian Hybrid Networks," *Applied Sciences*, doi:10.3390/app15179605, 2025.

[15] Cheng, J., Yang, Z.-Q., Cao, J., "Modeling Behavior Change for Multi-model At-Risk Students Early Prediction," *Proc. International Symposium on Educational Technology (ISET)*, IEEE, doi:10.1109/iset61814.2024.00020, 2024.

[16] Huang, K., Wang, C., "A Gaussian Process Classifier Integrating Meta-Heuristic Optimisers to Predict Student Performance," *International Journal of Information and Management Sciences*, doi:10.1504/ijims.2025.149393, 2025.

[17] Kayande, D., Kukreja, S., "An Integrated Multi-Modal Machine-Learning Framework for Real-Time Student Engagement Evaluation," *MethodsX*, doi:10.1016/j.mex.2025.103588, 2025.

[18] Liu, X., Li, X., Chen, J., "Adaptive Learning Path Planning System Based on Knowledge Graph and AI Integration," *Proc. International Conference on Educational Technology (ICET)*, IEEE, doi:10.1109/icet67421.2025.11380560, 2025.

[19] de Barba, P. G., Oliveira, E. A., English, N., "Development and Validation of a Learning Analytics Rubric for Self-Regulated Learning," *Educational Technology Research and Development*, doi:10.1007/s11423-025-10521-x, 2025.

[20] Uzun, Y., Suraworachet, W., Zhou, Q., "Engagement with Analytics Feedback and Its Relationship to Self-Regulated Learning Competence and Course Performance," *International Journal of Educational Technology in Higher Education*, doi:10.1186/s41239-025-00515-3, 2025.

[21] Butt, A. U. R., Ali, H., Asif, M., "Enhancing Student Management Through Hybrid Machine Learning and Rough Set Models," *IEEE Access*, doi:10.1109/access.2025.3565585, 2025.

[22] "CLST: Cold-Start Mitigation in Knowledge Tracing by Aligning a Generative Language Model as a Students' Knowledge Tracer," arXiv:2406.10296, 2024.

[23] "MAML-KT: Addressing the Cold-Start Problem in Knowledge Tracing for New Students via Few-Shot Model-Agnostic Meta-Learning," arXiv:2603.00137, 2026.

[24] "DKVMN&MRI: A New Deep Knowledge Tracing Model Based on DKVMN Incorporating Multi-Relational Information," *PLoS ONE*, doi:10.1371/journal.pone.0312022, 2024.

[25] "Personalised Forgetting Mechanism with Concept-Driven Knowledge Tracing," doi:10.1145/3810940, 2026.

[26] "Graph-based Knowledge Tracing for Personalized MOOC Path Recommendation," doi:10.69987/jacs.2025.51101, 2025.

[27] "Automatic Item Generation in Various STEM Subjects Using Large Language Model Prompting," *Computers and Education: Artificial Intelligence*, doi:10.1016/j.caeai.2024.100344, 2024.

[28] "Self-hosted Lecture-to-Quiz: Local LLM MCQ Generation with Deterministic Quality Control," arXiv:2603.08729, 2026.

[29] "Automated Generation and Tagging of Knowledge Components from Multiple-Choice Questions," doi:10.1145/3657604.3662030, 2024.

[30] "Automate Knowledge Concept Tagging on Math Questions with Large Language Models," arXiv:2403.17281, 2024.

[31] "Automatic Grading of Short Answers Using Large Language Models in Software Engineering Courses," *Proc. EDUCON*, IEEE, doi:10.1109/educon60312.2024.10578839, 2024.

[32] "Empirical Study of Large Language Models as Automated Essay-Scoring Tools in English Composition," arXiv:2401.03401, 2024.

[33] "Understanding Gaming the System by Analysing Self-Regulated Learning in Think-Aloud Protocols," doi:10.1145/3785022.3785025, 2026.

[34] "Advancing Online Assessment Integrity: Integrated Misconduct Detection via Internet-Protocol Analysis and Behavioural Analysis," *IEEE Access*, doi:10.1109/access.2024.3434608, 2024.

[35] "Personalised Language Learning Using Spaced Repetition Scheduling," doi:10.1007/978-3-031-98459-4_19, 2025.

[36] Jia, R., Jiang, Y.-H., Shen, Y., "SLAM: Sequential Learning Signal Modeling for Multi-Concept Knowledge Tracing," *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, doi:10.1109/icassp55912.2026.11464304, 2026.

[37] Su, S., Zeng, P., Kang, C., "Integrate Question Information With Learning Behavior for Knowledge Tracing," *IEEE Access*, doi:10.1109/access.2025.3543536, 2025.

[38] Nikolovski, V., Trajanov, D., Chorbev, I., "Advancing AI in Higher Education: A Comparative Study of LLM-Based Agents for Exam Question Generation," *Algorithms*, doi:10.3390/a18030144, 2025.

[39] Burke, C. M., "AI-Assisted Exam Variant Generation: A Human-in-the-Loop Framework for Automatic Item Creation," *Education Sciences*, doi:10.3390/educsci15081029, 2025.

[40] Alamoudi, S., Al Khuzayem, L. A., Jamal, A., "Optimizing Automated Question Generation for Educational Assessments," *Engineering, Technology & Applied Science Research (ETASR)*, doi:10.48084/etasr.10662, 2025.

[41] Lopes, A. M. M., de Magalhães Netto, J. F., "Large Language Models for Learning Technologies: Structural and Functional Evaluation of Automated Q-Matrix Generation," *IEEE Access*, doi:10.1109/access.2026.3683811, 2026.

[42] Hendrawan, R. A., de Alencar, R. S., Micheli, A., "Data-Driven Evaluation of LLM-Based Ontology Concept Extraction from Programming Learning Content," *Proc. Learning Analytics and Knowledge (LAK)*, ACM, doi:10.1145/3785022.3785106, 2026.

[43] Li, Z., Chen, L., Yi, J., "KeenKT: Knowledge Mastery-State Disambiguation for Knowledge Tracing," *Proc. AAAI Conference on Artificial Intelligence*, doi:10.1609/aaai.v40i23.39004, 2026.

[44] Jukiewicz, M., "A Systematic Comparison of Large Language Models for Automated Assignment Assessment in Programming Education," *Computers and Education Open*, doi:10.1016/j.caeo.2026.100364, 2026.

[45] Senanayake, C., Asanka, D., "Rubric-Based Automated Short Answer Scoring Using Large Language Models," *Proc. International Conference on Smart Computing and Systems Engineering (SCSE)*, IEEE, doi:10.1109/scse61872.2024.10550624, 2024.

[46] Baral, S., Worden, E., Lim, W.-C., "Automated Assessment in Math Education: A Comparative Analysis of LLMs for Open-Ended Responses," *Proc. Educational Data Mining (EDM)*, doi:10.5281/zenodo.12729932, 2024.

[47] Erdem, B., Karabatak, M., "Cheating Detection in Online Exams Using Deep Learning and Machine Learning," *Applied Sciences*, doi:10.3390/app15010400, 2025.

[48] "LLM-Generated Retrieval Practice: Linking Automatic Question Generation to Spaced Retrieval," arXiv:2507.05629, 2025.